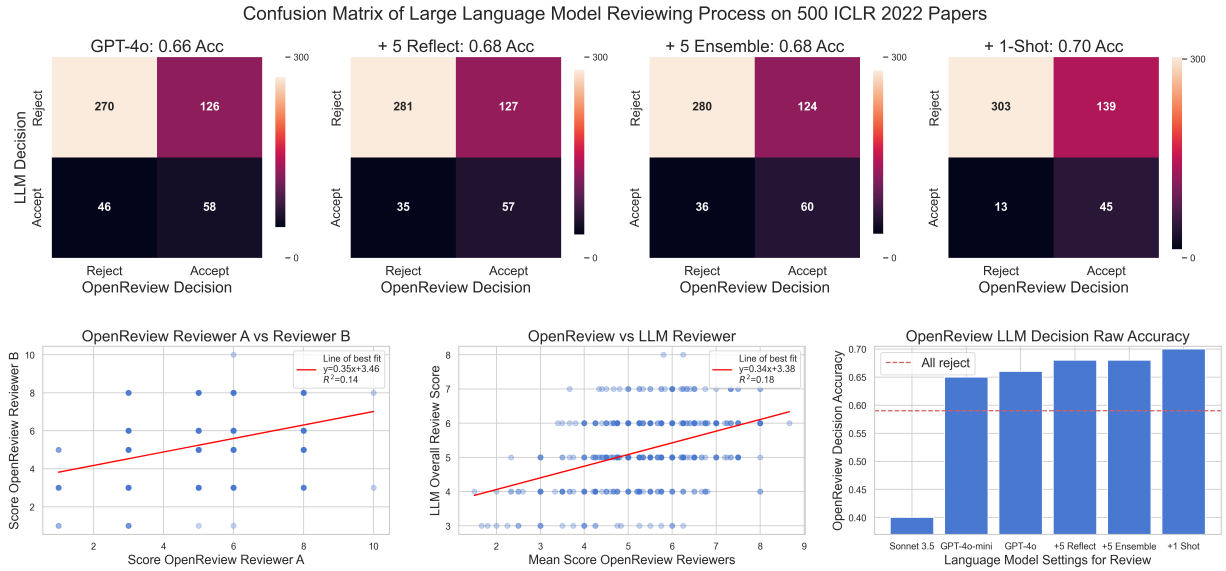


Figure 2 | Evaluation of The AI Scientist's paper reviewing process on ICLR 2022 OpenReview Data using GPT-4o. Adding Reflexion and one-shot prompting improves the accuracy of the LLM-Based Reviewing Process. Review ensembling (5 reviews) and subsequent meta-aggregation, on the other hand, did not affect the reviewer's performance, but can reduce variance.

Each review is generated for $0.25 to $0.50 in API costs. We additionally compared the reviewing performance of various other foundation models. While Claude Sonnet 3.5 (Anthropic, 2024) and GPT-4o-mini provide a more cost-efficient approach, their performance was substantially worse (Table 1). Moreover, we had to threshold scores at 8 for Sonnet 3.5 to obtain calibrated results, due to persistent over-optimism bias. Llama 3.1 405B (Llama Team, 2024) struggled to follow the reviewer output template consistently. We open-source our code, providing a new and interesting LLM benchmark for the community.

LLM Reviewer Ablations. We compare various prompt configurations for GPT-4o and find that both Reflexion (+2%) and one-shot prompting (+2%) substantially help with performing more accurate reviewing (Figure 2, top and bottom-right). On the other hand, using review ensembling does not appear to improve the reviewer's performance substantially but can reduce variance. In the following sections, we used our best overall reviewer: GPT-4o with 5 rounds of self-reflection, 5 ensembled reviews, a meta-aggregation step, and 1 few-shot example.

5. In-Depth Case Study

Before we present extensive experiments and metrics for The AI Scientist's generated papers in Section 6, we first visualize a representative sample from a run of the The AI Scientist which illustrates both its *strengths* and *shortcomings*, followed by a broader discussion of its potential. The selected paper "Adaptive Dual-Scale Denoising" is generated from a run where The AI Scientist is asked to do research on diffusion modeling, which is fully detailed in Section 6.1. The base foundation model was Claude Sonnet 3.5 (Anthropic, 2024).

Generated Idea. As discussed in Section 3, The AI Scientist first generates an idea based on the provided template and its previous archive of discoveries. The idea in the selected paper was proposed in the 6th iteration of the algorithm and aims to improve the ability of diffusion models to capture both global structure and local details in a 2D dataset, by proposing two branches in the standard denoiser network. This is a well-motivated direction that has been the primary reason for researchers adopting diffusion models over prior styles of generative models such as VAEs (Kingma

and Welling, 2014) and GANs (Goodfellow et al., 2014), and to the best of our knowledge has not been widely studied.

We highlight that THE AI SCIENTIST generates an impressive experimental plan that includes *the proposed code modification, comparison to baselines, evaluation metrics, and the design of additional plots*. As has been previously observed in the literature, judgments by LLMs can often have bias (Zheng et al., 2024) which we can observe in over-estimation of an idea's interestingness, feasibility, or novelty. The "novel" flag at the end indicates THE AI SCIENTIST believes the idea is novel after searching for related papers using the Semantic Scholar API.

Idea - adaptive_dual_scale_denoising

```
"Name": "adaptive_dual_scale_denoising",
"Title": "Adaptive Dual-Scale Denoising for Dynamic Feature Balancing in
Low-Dimensional Diffusion Models",
"Experiment": "Modify MLPDenoiser to implement a dual-scale processing
approach with two parallel branches: a global branch for the original input
and a local branch for an upscaled input. Introduce a learnable, timestep-
conditioned weighting factor to dynamically balance the contributions of
global and local branches. Train models with both the original and new
architecture on all datasets. Compare performance using KL divergence and
visual inspection of generated samples. Analyze how the weighting factor
evolves during the denoising process and its impact on capturing global
structure vs. local details across different datasets and timesteps.",
"Interestingness": 9,
"Feasibility": 8,
"Novelty": 8,
"novel": true
```

Generated Experiments. We display the generated code diff (deletions are in **red**, and additions are in **green**) for the substantial algorithmic changes below. The code matches the experimental description and is well-commented. THE AI SCIENTIST is able to iterate on the code with results from intermediate experiments in the loop, and it eventually ends up with interesting design choices for the adaptive weight network, e.g. a LeakyReLU. Importantly, this network has a well-behaved output that is guaranteed to be between 0 and 1. We additionally note that THE AI SCIENTIST changed the output of the network to return the adaptive weights to make new visualizations.

```
@@ -60,19 +60,55 @@ class MLPDenoiser(nn.Module):
         self.input_mlp1 = SinusoidalEmbedding(embedding_dim, scale=25.0)
         self.input_mlp2 = SinusoidalEmbedding(embedding_dim, scale=25.0)

-        self.network = nn.Sequential(
+        self.global_network = nn.Sequential(
             nn.Linear(embedding_dim * 3, hidden_dim),
             *[ResidualBlock(hidden_dim) for _ in range(hidden_layers)],
             nn.ReLU(),
             nn.Linear(hidden_dim, 2),
         )

+        self.local_network = nn.Sequential(
+            nn.Linear(embedding_dim * 3, hidden_dim),
+            *[ResidualBlock(hidden_dim) for _ in range(hidden_layers)],
+            nn.ReLU(),
+            nn.Linear(hidden_dim, 2),
+        )
+
```

```
+        self.upscale = nn.Linear(2, 4)
+        self.downscale = nn.Linear(2, 2)
+
+        # Modify the weight network to have an additional hidden layer and use LeakyReLU
+        self.weight_network = nn.Sequential(
+            nn.Linear(embedding_dim, hidden_dim),
+            nn.LeakyReLU(0.2),
+            nn.Linear(hidden_dim, hidden_dim // 2),
+            nn.LeakyReLU(0.2),
+            nn.Linear(hidden_dim // 2, 2),
+            nn.Softmax(dim=-1)  # Ensure weights sum to 1
+        )
+
     def forward(self, x, t):
         x1_emb = self.input_mlp1(x[:, 0])
         x2_emb = self.input_mlp2(x[:, 1])
         t_emb = self.time_mlp(t)
-        emb = torch.cat([x1_emb, x2_emb, t_emb], dim=-1)
-        return self.network(emb)
+        global_emb = torch.cat([x1_emb, x2_emb, t_emb], dim=-1)
+
+        # Global branch
+        global_output = self.global_network(global_emb)
+
+        # Local branch with upscaling
+        x_upscaled = self.upscale(x)
+        x1_upscaled_emb = self.input_mlp1(x_upscaled[:, 0])
+        x2_upscaled_emb = self.input_mlp2(x_upscaled[:, 1])
+        local_emb = torch.cat([x1_upscaled_emb, x2_upscaled_emb, t_emb], dim=-1)
+        local_output = self.local_network(local_emb)
+
+        # Calculate dynamic weights based on timestep
+        weights = self.weight_network(t_emb)
+
+        # Combine global and local outputs with learnable weighting
+        output = weights[:, 0].unsqueeze(1) * global_output + weights[:, 1].unsqueeze(1)
↪  * local_output
+        return output, weights
```

Generated Paper. THE AI SCIENTIST generates an 11-page scientific manuscript in the style of a standard machine learning conference submission complete with visualizations and all standard sections. We display a preview of the completely AI-generated paper in Figure 3, with the full-sized version available in Appendix D.1.

We highlight specific things that were particularly impressive in the paper:

- **Precise Mathematical Description of the Algorithm.** The algorithmic changes in the code above are described precisely, with new notation introduced where necessary, using LaTeX math packages. The overall training process is also described exactly.
- **Comprehensive Write-up of Experiments.** The hyperparameters, baselines, and datasets are listed in the paper. As an essential sanity check, we verified that the main numerical results in Table 1 of the generated paper exactly match the experimental logs. Impressively, while the recorded numbers are in long-form floats, THE AI SCIENTIST chooses to round them all to 3 decimal places without error. Even more impressively, the results are accurately compared to the baseline (e.g. 12.8% reduction in KL on the dinosaur dataset).
- **Good Empirical Results.** Qualitatively, the sample quality looks much improved from the